

开发流程说明书

基于 ESP32-C3 与 MLX90640 的热成像相机系统设计

面向 “微机接口与技术 / 微机原理” 课程期末大作业展示

项目项	内容
维护者	rdfrdream
仓库地址	https://github.com/1909899270h-spec/esp32c3-mlx90640-thermal-camera
主控平台	ESP32-C3 DevKit, ESP-IDF + PlatformIO 工程
核心传感器	MLX90640 32x24 热红外阵列, I2C 地址 0x33
文档定位	记录从初期计划到软件、硬件、调试、展示和开源发布的完整开发流程

开发主线

MLX90640 热红外阵列 -> I2C 总线 -> ESP32-C3 -> WiFi/AP + HTTP Server -> 浏览器热图 -> CSV 数据记录。该链路同时支持真实硬件采集和本地仿真演示，使传感器采集、网页可视化与数据记录能够独立验证。

1. 项目背景与开发目标

本项目来源于微机接口与技术课程大作业。最初目标不是单纯点亮传感器，而是完成一套能够体现“微机接口、嵌入式采集、无线通信、网页可视化和数据记录”的小型系统。项目选择 ESP32-C3 作为主控，是因为它可以同时承担控制器、WiFi 通信节点和轻量级 Web Server 的角色。

目标类别	具体目标	对应实现
采集目标	读取 32x24 温度阵列，形成 768 点温度矩阵	MLX90640 + I2C，总线地址 0x33
通信目标	摆脱串口单机查看，支持手机或电脑无线访问	ESP32-C3 SoftAP / HTTP Server
展示目标	将温度矩阵转换为接近热像仪的热图	网页 Canvas 绘图、色温条、平滑处理
记录目标	按时间顺序保留温度帧，便于分析与答辩展示	浏览器端记录与 CSV 下载
调试目标	在硬件链路未完全稳定时仍能展示软件流程	Python/浏览器仿真数据链路

2. 初期计划拆解

仓库 README 中的初期计划可以概括为一条端到端数据链路：传感器负责采集，ESP32-C3 负责接口与服务，浏览器负责显示和导出。为了便于开发执行，该链路被拆成硬件、固件、前端、仿真和发布五类任务。

MLX90640 热红外阵列	I2C GPIO4/GPIO5	ESP32-C3 主控处理	WiFi/AP HTTP 服务	浏览器网页 热图与 CSV
-------------------	--------------------	------------------	--------------------	------------------

数据方向：温度矩阵 -> JSON 接口 -> Canvas 热图 -> CSV 记录

任务域	初期设想	开发后形成的工程内容
硬件连接	用面包板替代焊接，先验证 SDA/SCL/VDD/GND	明确 GPIO4/GPIO5、3.3V/GND、5kΩ 上拉、10uF 去耦
固件采集	用 ESP32-C3 读取 MLX90640 温度矩阵	实现 I2C 初始化、EEPROM 读取、参数解析、帧采集任务
无线网页	ESP32 建立热点，手机连入后打开网页	SoftAP 生成 IRCAM-XXXX 热点，HTTP 服务提供网页与数据接口

数据展示	网页显示热图并能下载数据	Canvas 热图、色温条、导入数据、开始检测、CSV 下载、全屏
备选演示	硬件不稳定时仍能完成课程展示	本地 mock_thermal_server.py 与浏览器端平滑仿真热源

3. 系统总体开发流程

开发流程采用“先打通链路、再优化体验、最后补齐调试与展示材料”的顺序。这样做的好处是每个阶段都有可验证输出，避免硬件问题阻塞全部软件展示。

阶段	开发动作	阶段输出	验证方式
1 需求定义	明确课程展示要求、硬件限制和最终交互方式	系统目标、数据链路、演示边界	是否能解释项目价值和技术路线
2 器件选型	比较 C51、STM32、ESP32-C3，并确认 MLX90640	主控与传感器方案	是否满足 WiFi Web 与 I2C 采集
3 工程搭建	建立 PlatformIO + ESP-IDF 工程，配置分区和 SPIFFS	可编译工程	pio run 通过
4 固件开发	实现 I2C、传感器初始化、帧采集、HTTP 接口	ESP32 端数据服务	串口日志、/status、/i2c-scan
5 前端开发	完成热图、色温条、控制按钮、CSV 记录	浏览器可视化界面	本地或 ESP32 网页访问
6 仿真补充	加入平滑运动热源，形成可靠展示链路	无硬件演示能力	导入数据、录屏展示
7 发布归档	整理 README、LICENSE、NOTICE，上传 GitHub	开源仓库与文档	远端文件可读取

4. 硬件设计与接线流程

硬件开发采取低风险验证路线：先用面包板和杜邦线完成连接，确认 ESP32-C3 能被电脑识别，再进行 I2C 总线识别，最后观察网页数据是否变化。由于购买的是裸 MLX90640 传感器，引脚机械稳定性是调试重点。

MLX90640 引脚	ESP32-C3 引脚	用途	注意事项
VDD	3.3V	传感器供电	避免接 5V，裸传感器更需要注意供电稳定
GND	GND	共地	ESP32 与传感器必须共地，否则 I2C 电平无参考
SDA	GPIO4	I2C 数据线	建议 5kΩ 上拉到 3.3V，避免线松导致识别失败
SCL	GPIO5	I2C 时钟线	建议 5kΩ 上拉到 3.3V，保持短线和稳定接触

硬件附加元件作用

5kΩ 上拉电阻用于保证 SDA/SCL 在空闲时为高电平，是 I2C 总线可靠识别的重要条件。10uF 电容并联在 VDD 与 GND 之间，用于抑制供电波动，降低传感器初始化或读取帧数据时的异常概率。

检查层级	判断方法	通过标志
电源层	ESP32-C3 Type-C 上电，电脑设备管理器识别 COM 口	COM5 或类似串口出现，开发板电源灯正常
接线层	核对 GPIO4/GPIO5、3.3V/GND、上拉电阻和电容位置	无短路、无 5V 误接、传感器端接触稳定
总线层	访问 /i2c-scan 或查看串口日志	扫描到 0x33
数据层	点击网页开始检测，观察 min/max/热图变化	温度范围随热源变化且无连续读数错误

5. 开发环境与工程配置流程

软件开发选择 VS Code + PlatformIO。VS Code 提供统一编辑入口，PlatformIO 负责工程管理、依赖下载、编译、烧录、串口监视和 SPIFFS 文件系统上传。ESP-IDF 是底层框架，提供 I2C、WiFi、HTTP Server、FreeRTOS 等能力。

工具/文件	作用	本项目中的体现
VS Code	代码编辑与任务入口	打开 Thermal_Imaging.code-workspace
PlatformIO	工程配置、编译、烧录、串口监视	platformio.ini 定义板卡、框架、速度和 SPIFFS
ESP-IDF	ESP32 官方开发框架	driver/i2c、esp_wifi、esp_http_server、FreeRTOS
partitions.csv	Flash 分区配置	为网页文件系统 SPIFFS 预留空间
data/web_script.js	前端脚本资源	通过 uploadfs 上传到 ESP32 的 SPIFFS

```
[env:esp32-c3-devkitm-1]
platform = espressif32@6.9.0
board = esp32-c3-devkitm-1
framework = espidf
monitor_speed = 115200
upload_speed = 921600
board_build.flash_size = 4MB
board_build.partitions = partitions.csv
board_build.filesystem = spiffs
```

6. 固件开发流程

固件开发围绕两条线进行：一条是 MLX90640 采集链路，另一条是 WiFi Web 服务链路。前者要求总线稳定和帧数据正确，后者要求手机或电脑能够通过局域网访问 ESP32 提供的数据接口。

固件模块	关键实现	输出/接口
I2C 初始化	GPIO4 作为 SDA，GPIO5 作为 SCL，400kHz 主机模式	I2C 驱动安装成功，支持地址探测
MLX90640 初始化	DumpEE、ExtractParameters、SetRefreshRate、SetChessMode	传感器参数有效，刷新率设置完成
帧采集任务	周期读取 frameData，计算 768 个温度点	mlx90640_temperatures 数组
JSON 数据接口	将温度数组序列化为二维 JSON 数组	/thermal-data
状态调试接口	返回 GPIO、初始化状态、错误计数、I2C 探测结果	/status、/i2c-scan

```
#define I2C_MASTER_SCL_IO    GPIO_NUM_5
#define I2C_MASTER_SDA_IO    GPIO_NUM_4
#define I2C_MASTER_FREQ_HZ    400000
const uint8_t MLX90640_I2C_ADDR = 0x33;

// 初始化顺序：
// 1. i2c_master_init()
// 2. MLX90640_DumpEE()
// 3. MLX90640_ExtractParameters()
// 4. MLX90640_SetRefreshRate()
// 5. MLX90640_SetChessMode()
```

7. WiFi Web Server 与局域网访问流程

ESP32-C3 在 AP 模式下本身就是一个小型服务器。手机或电脑连接 ESP32 创建的 WiFi 热点后，浏览器访问 192.168.4.1，即可请求 ESP32 上运行的 HTTP 服务。这个地址属于局域网私有地址，因此切换到其他网络后不能直接访问。

步骤	动作	说明
1	ESP32-C3 启动 SoftAP	热点名由 MAC 后两字节生成，形式类似 IRCAM-XXXX

2	手机/电脑连接热点	接入后设备与 ESP32 处于同一局域网
3	浏览器访问 192.168.4.1	请求 ESP32 的 HTTP Server
4	网页请求数据接口	前端周期访问 /thermal-data 或调试访问 /status
5	Canvas 绘制热图	将 32x24 温度矩阵映射为彩色热成像画面

为什么不是公网网页

ESP32 的 192.168.4.1 是局域网地址，只在连接 ESP32 热点或同一路由器网络时有效。它不是互联网公网服务器，因此不需要云服务器，也不会被外部网络直接访问。

8. 前端热图与数据交互开发流程

前端网页承担展示层任务。真实检测模式读取 ESP32 的 /thermal-data；导入数据模式使用浏览器端平滑仿真热源，用于硬件调试期间录制演示视频。两种模式共用同一套热图绘制、色温条和 CSV 记录逻辑。

功能	实现思路	展示价值
热图绘制	Canvas 将 32x24 温度点插值放大，并用色表映射颜色	直观看看到高温区域和运动趋势
色温条	根据当前帧 min/max 更新颜色范围	让颜色与温度含义对应
开始检测	请求 ESP32 /thermal-data	体现真实硬件采集链路
导入数据	生成变化速率受限的平滑热源	保证演示时画面稳定、可信
下载 CSV	按帧记录时间、来源和 768 点温度	便于后续数据分析和答辩说明
全屏	浏览器进入全屏模式	方便全屏观察、录屏保存和现场说明

```
const srcWidth = 32;
const srcHeight = 24;
const frameSize = srcWidth * srcHeight;
```

// 关键前端流程：

```
// fetch('/thermal-data') -> JSON 温度数组
// smoothGrid() -> 插值平滑
// colorStops() -> 颜色映射
// drawThermalFrame() -> Canvas 热图
// downloadCsv() -> 导出数据记录
```

9. 仿真与演示链路开发流程

考虑到裸传感器接线松动、面包板空间有限、I2C 上拉和供电去耦可能影响稳定性，项目加入仿真演示链路。这一链路不替代真实采集，但能完整展示网页界面、热源运动、色温条和 CSV 导出流程。

仿真环节	开发处理	边界说明
热源模型	生成运动的高温区域，并加入背景梯度、噪声和帧间限制	用于模拟热像仪观感，不代表真实温度
变化速率	限制相邻帧温度变化，避免画面跳变	符合热成像采样的连续性
网页复用	仿真数据进入同一热图绘制流程	验证前端不依赖具体数据来源
本地服务	tools/mock_thermal_server.py 提供本地演示页面	用于电脑录屏或无传感器调试

```
python tools/mock_thermal_server.py

# 或双击：
Run_Mock_Thermal_Web.cmd

# 浏览器访问：
http://127.0.0.1:8000
```

10. 编译、烧录与调试流程

烧录过程分为主程序烧录和网页文件系统烧录。主程序包含 ESP32 固件逻辑；SPIFFS 文件系统包含网页资源。两者都完成后，浏览器端界面才能和 ESP32 数据接口对应起来。

操作	命令	用途
编译固件	pio run -e esp32-c3-devkitm-1	检查 ESP-IDF 工程是否能完整构建

烧录固件	<code>pio run -e esp32-c3-devkitm-1 -t upload</code>	把主程序写入 ESP32-C3
上传网页	<code>pio run -e esp32-c3-devkitm-1 -t uploadfs</code>	把 data 目录中的网页资源写入 SPIFFS
串口监视	<code>pio device monitor -e esp32-c3-devkitm-1</code>	查看 COM 日志、I2C 初始化和 WiFi 地址

烧录卡在 Connecting... 的处理

按住 BOOT/FLASH，再轻按 RST/RESET；当终端出现 Writing at... 后松开 BOOT/FLASH。若出现 PermissionError 或 COM 口拒绝访问，通常是串口监视器仍占用端口，需要关闭 monitor 后重试。

调试层	观察点	正常表现	异常处理
硬件层	设备管理器 COM 口、开发板电源灯	识别 COM5 或类似端口	检查 Type-C 数据线、驱动和端口占用
总线层	/i2c-scan、串口日志	found 包含 0x33	检查 SDA/SCL、上拉电阻、供电和接触松动
数据层	/thermal-data、网页 min/max	返回 768 点温度数组	检查传感器初始化状态和 I2C 错误计数
展示层	网页热图、色温条、CSV	画面随热源变化并可导出	切换导入数据模式排除前端问题

11. 当前成果与验收标准

当前软件链路已经整理完成并上传到 GitHub，包含固件、网页、仿真、README、LICENSE 和 NOTICE。真实 MLX90640 实时采集链路已经具备代码和接线方案，但仍建议在最终展示前继续做稳定性验证。

成果项	状态	验收依据
PlatformIO 工程	已完成	可以编译 ESP32-C3 固件并生成 SPIFFS 文件系统
Web Server	已完成	浏览器可访问热成像页面
网页热图	已完成	具备热图、色温条、开始检测、导入数据、CSV 下载、全屏

仿真演示	已完成	本地服务器和浏览器端数据均可用于展示
硬件采集	稳定性验证中	以 /i2c-scan 是否识别 0x33 和 /thermal-data 是否持续返回正常帧为准
开源发布	已完成	GitHub 仓库 main 分支包含完整整理后的代码和文档

验收维度	最低要求	推荐展示方式
系统说明	能解释从传感器到网页热图的数据链路	结合总体流程图和接线表说明
软件验证	能打开网页并看到动态热图	使用导入数据或真实检测录屏
硬件调试	能说明 0x33 地址识别与三层排错逻辑	展示 /status 或 /i2c-scan 思路
工程规范	代码、README、许可证和说明文件齐全	展示 GitHub 仓库

12. GitHub 开源整理流程

项目发布阶段将软件内容从临时开发目录整理成独立开源仓库。整理重点是去除无关中间产物，保留可复现的源代码、配置、脚本和文档。

整理对象	保留/处理	原因
src/main.c	保留	ESP32-C3 固件主逻辑
data/web_script.js	保留	网页热图与交互脚本
lib/MLX90640	保留并保留原注释	传感器参数解析与温度计算库
tools/mock_thermal_server.py	保留	用于演示和前端调试
outputs/deliverables	默认忽略	避免将 PPT、PDF、编译产物混入源码仓库
README/NOTICE/LICENSE	新增/完善	说明用途、署名、开源协议和上游来源

```
git add -A
git commit -m "Clean repository for public release"
git push -u origin main
```

仓库地址：
<https://github.com/1909899270h-spec/esp32c3-mlx90640-thermal-camera>

13. 工程化升级方向与可行性分析

后续优化可围绕“感知更准、响应更快、访问更远、判断更智能”展开。结合现有 ESP32-C3、MLX90640 和网页数据链路，以下方向具备可实施基础。

方向	具体实现	可行性判断
温度极值报警	设置最高温、最低温或区域均值阈值；超限后驱动 LED、蜂鸣器或网页报警提示。	可行性最高。ESP32-C3 GPIO 足够，软件只需计算 min/max/avg 并判断阈值。
远程访问与云端化	通过 MQTT/HTTP/WebSocket 将数据上传到服务器，由公网网页或手机端查看。	可行。不要直接暴露 192.168.4.1，建议 ESP32 主动上传，避免公网 IP 和端口映射依赖。
红外自动识别	在温度矩阵上做热点分割、区域追踪、异常温升判断，再尝试轻量级分类。	可行但需分阶段。32x24 分辨率适合热点和趋势识别，不适合高精度图像识别。
温度校准系统	加入参考温度源或标准温度计，建立多点校准曲线，并保存 offset、scale、发射率等参数。	可行且必要。没有参考源只能做相对校准；加入标准温度后可提高测量可信度。
硬件可靠性	设计转接板或 PCB，固定 MLX90640，增加可靠接口、去耦电容、外壳和稳定供电。	可行。可降低裸传感器接触松动、I2C 读写失败和供电波动。
数据记录增强	增加本地数据库、时间戳同步、历史曲线、阈值事件日志和 CSV/JSON 导出。	可行。现有 CSV 逻辑已经具备基础，可继续扩展长期记录和事件追踪。

可行性结论

方案总体可行。建议优先做阈值报警和接线可靠性优化；随后加入校准参数和参考温度验证；再扩展云端远程访问；最后推进红外自动识别算法。

附录 A：关键文件索引

路径	说明
platformio.ini	PlatformIO 工程、板卡、框架、烧录速度和 SPIFFS 配置
src/main.c	I2C、MLX90640、WiFi AP、HTTP Server 和调试接口实现
data/web_script.js	网页热图、色温条、仿真导入、CSV 下载和全屏逻辑
lib/MLX90640	MLX90640 参数解析、帧数据读取和温度计算
tools/mock_thermal_server.py	本地演示服务器，支持无硬件时的动态热图展示
README.md	仓库说明、接线、烧录、使用和调试入口
NOTICE.md / LICENSE	署名、来源说明和开源协议